★ LEVEL 2 · 연결

Claude Code를 볼트의 사서로

같은 폴더를 Claude Code로 열고, 정리는 명령 하나로 맡긴다

Level 2 · 연결 — 키트 설치부터 /inbox-triage까지

준비물: Level 1에서 만든 볼트, Claude 유료 요금제(Pro 이상).
이 강의가 끝나면 명령 몇 개로 볼트를 정리할 수 있다.

Level 2가 전하는 세 가지 메시지

정리는 Claude가 하고, 사람은 계획을 보고 승인한다

1

Claude Code는 내 폴더의 파일을 직접 읽고 고친다

복사·붙여넣기가 없다 — 볼트 폴더를 열어 주면 그 안의 노트를 바로 다룬다.

2

규칙은 CLAUDE.md, 반복 작업은 명령 파일에 적어 둔다

한 번 적어 두면 매번 같은 방식으로 일한다 — 같은 설명을 반복하지 않는다.

3

Claude는 계획을 먼저 보여 주고, 사람이 승인한다

옮기기 전에 계획표를 보고, 잘못되면 Git 기록으로 되돌린다.

오늘의 결과물: 키트가 설치된 볼트 + Inbox 노트를 명령 하나로 분류

Claude Code는 채팅과 달리 폴더 안에서 일한다

Claude Code — 내 컴퓨터의 폴더를 열어 파일을 읽고 쓰는 Claude

채팅 (Claude 대화창)



사람이 파일을 옮겨 나른다

손이 많이 간다

Claude Code

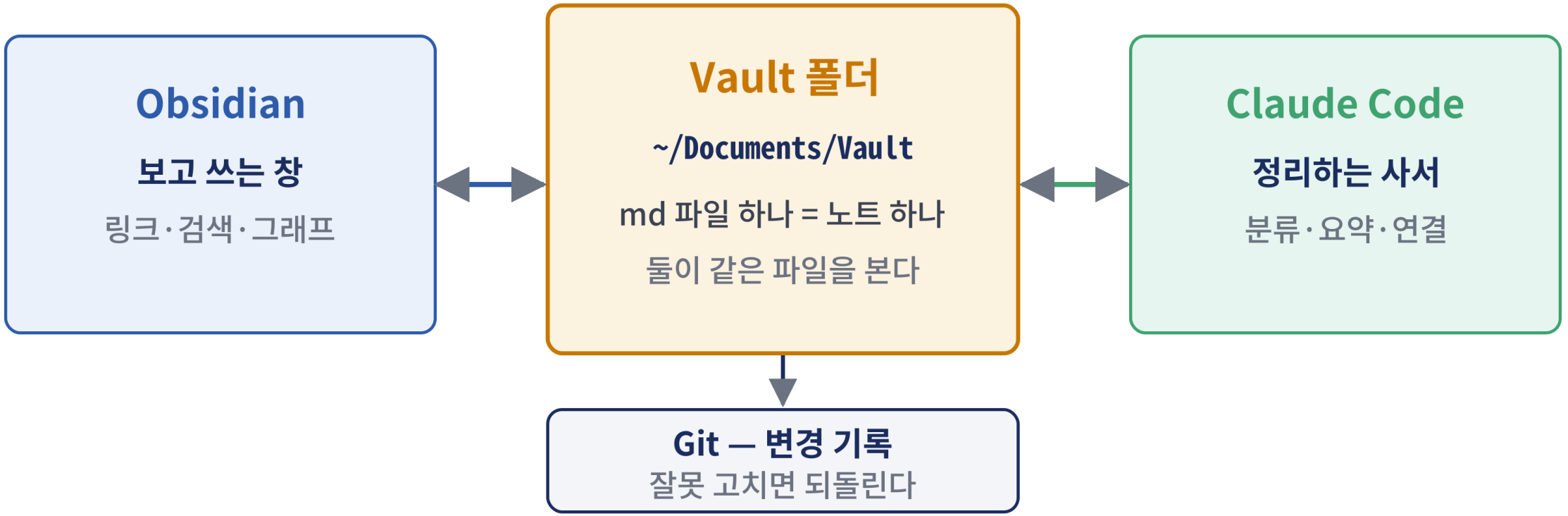


Claude가 폴더 안에서 일한다

사람은 확인·승인만

옵시디언과 Claude Code는 같은 폴더를 함께 본다

Claude가 고친 노트는 옵시디언 화면에 바로 나타난다



시작 전에 요금제·앱·Git 세 가지를 확인한다

3-1 과정 Claude 설치 가이드(2026년 10월) 기준

항목	필요한 것	확인하는 곳
요금제	Pro · Max · Team · Enterprise (무료는 대화만 가능)	claude.ai 설정
앱	Claude 데스크톱 앱 — 안에 Code가 들어 있다	claude.com/download
Node.js	필요 없다 — 앱에 이미 들어 있다	—
Git	Mac은 대부분 기본 설치, Windows는 설치 권장	git-scm.com

회사망에서 로그인이 멈추면 claude.ai · api.anthropic.com 허용을 정보화 부서에 요청한다

01

LEVEL 2 · 연결

볼트를 Claude Code로 열기

폴더 하나를 고르면 연결이 끝난다

데스크톱 앱의 Code 화면에서 볼트 폴더를 고른다

Code → Local → Select folder 순서, 처음에는 권한 모드를 Manual로 둔다

① 앱 실행

로그인

Claude 데스크톱 앱을 열고 유료 계정으로 로그인한다

② Code 열기

왼쪽 화면

대화가 아니라 Code 화면을 고른다

③ 폴더 고르기

Local → Select folder

~/Documents/Vault 폴더를 선택한다

④ Manual 모드

권한 모드

바뀔 내용을 하나씩 보고 승인한다

익숙해지면 바꾼다

VS Code를 쓰는 사람은 Claude Code 확장(anthropic.claude-code)으로 같은 폴더를 열어도 된다

첫 대화는 폴더를 설명하게 하는 것으로 시작한다

바로 고치게 하지 말고, 읽고 설명하게 해서 연결을 확인한다

[입력] 프롬프트 · Claude Code

```
이 폴더가 어떤 곳인지 처음 보는
사람도 알 수 있게 설명해 줘.
폴더 구조를 트리로 보여 주고,
아무 파일도 고치지 마.
```

[출력] 결과 · Claude의 답 (예시)

```
이 폴더는 옵시디언 볼트입니다.
PARA 방식으로 나뉘어 있습니다.
Vault/
 ├──── 00_Inbox      (노트 0개)
 ├──── 01_Projects   (노트 1개)
 ├──── 03_Resources  (노트 1개)
 └──── 90_Daily      (노트 1개)
```

Level 1에서 만든 노트 개수가 맞게 나오면 연결 성공

Git은 폴더의 변경 기록을 남겨 되돌리게 한다

Git — 저장할 때마다 폴더의 '사진'을 찍어 두는 버전 관리 도구

커밋(commit) = 지금 상태를 사진 한 장으로 저장 · 되돌리기 = 예전 사진으로 돌아가기

왜 볼트에 Git인가

- Claude는 여러 파일을 한 번에 바꾼다
- 잘못돼도 한 번에 원래대로 돌린다
- '최종_v3' 같은 파일명이 필요 없다

명령은 몰라도 된다

- Claude에게 말로 시킨다
- '백업해 줘' '되돌려 줘'면 된다
- Git 명령은 Claude가 실행한다

Git 설정도 Claude에게 말로 맡긴다

명령을 외우지 않고 원하는 결과를 설명한다

[입력] 프롬프트 · Claude Code

```
이 볼트를 Git으로 백업할 수 있게
설정해 줘.
- .obsidian/workspace*, .DS_Store 제외
- 첫 커밋 메시지: init second brain
실행 전에 무엇을 할지 먼저 보여 줘.
```

[출력] 결과 · Claude가 실행한 일

```
1. git init
2. .gitignore 생성 (제외 2줄)
3. git add -A
4. git commit -m "init second brain"

→ 커밋 1개, 파일 6개가 기록되었습니다.
```

Windows에서 'Git is required'가 뜨면 git-scm.com에서 Git을 설치하고 앱을 다시 연다

02

LEVEL 2 · 키트

세컨드 브레인 키트 설치

규칙 파일 1개 + 명령 9개 + 템플릿 16개

키트는 규칙·명령·템플릿 세 묶음이다

second-brain-kit-v2.zip — 볼트 루트(맨 위 폴더)에 풀어 넣는다

묶음	들어 있는 곳	하는 일
규칙	CLAUDE.md	Claude가 지킬 볼트 규칙
명령	.claude/commands/ (9개)	/inbox-triage 같은 반복 작업
템플릿	98_Templates/ (16개)	노트 종류별 빈 양식
대시보드	02_Areas/_dashboards/home.md	현황판 (Level 3)
폴더	PARA 폴더 + 90_Daily 등	Level 1 폴더에 더해진다

숨김 파일 .claude/ · .gitignore는 직접 복사하면 빠지기 쉽다 (Mac: Cmd+Shift+.)

키트 복사는 Claude에게 맡긴다

zip을 00_Inbox에 넣고 아래처럼 시킨다 — 숨김 파일까지 빠짐없이 들어간다

[입력] 프롬프트 · Claude Code

```
00_Inbox에 있는 second-brain-kit-v2.zip을
볼트 루트에 풀어 줘.
- 이미 있는 내 노트·폴더는 덮어쓰지 말 것
- .claude/ 와 .gitignore 도 포함
- 끝나면 추가된 파일 목록을 보여 주고
  커밋해 줘.
```

[출력] 결과 · 추가된 파일

```
+ CLAUDE.md
+ .claude/commands/    (9개)
+ 98_Templates/        (16개)
+ 02_Areas/_dashboards/home.md
= 기존 노트 5개는 그대로

커밋: install second brain kit v2
```

설치 후 Claude Code를 다시 열면 / 를 쳤을 때 명령 목록이 보인다

CLAUDE.md는 Claude가 매번 먼저 읽는 규칙서다

Claude Code는 연 폴더의 CLAUDE.md를 자동으로 읽고 그대로 따른다

[예시] CLAUDE.md 일부

```
## 3. 노트 작성 규칙
- 파일명: 영어 소문자 kebab-case
- 수식: 항상 LaTeX
## 4. 금지 사항
- .obsidian/ 은 건드리지 않는다
- 노트를 삭제하지 않는다
## 5. 작업 원칙
- 5개 이상 옮기면 계획표 먼저
```

내 규칙으로 고친다

- 용어·문체 규칙을 더한다
- 폴더 이름을 바꿨으면 반영한다
- 말로 시킨다: 'CLAUDE.md에 ○○ 규칙 추가해 줘'

명령 파일은 자주 하는 일을 한 단어로 부르게 한다

Custom command — .claude/commands/의 파일 이름이 곧 명령 이름이다

명령	언제	하는 일
/inbox-triage	주 1~2회	Inbox 노트를 PARA 폴더로 분류
/atomize 노트	긴 자료를 받았을 때	개념·케이스 노트로 쪼갠다
/link 노트	새 노트를 쓴 뒤	관련 노트와 양방향으로 잇는다
/ask 질문	자료를 찾을 때	내 노트만 근거로 답한다
/weekly-review	주말	한 주를 점검한다 (Level 3)

나머지 /meeting · /new-project · /status · /retention 은 Level 3에서 다룬다

03

LEVEL 2 · 명령 실습

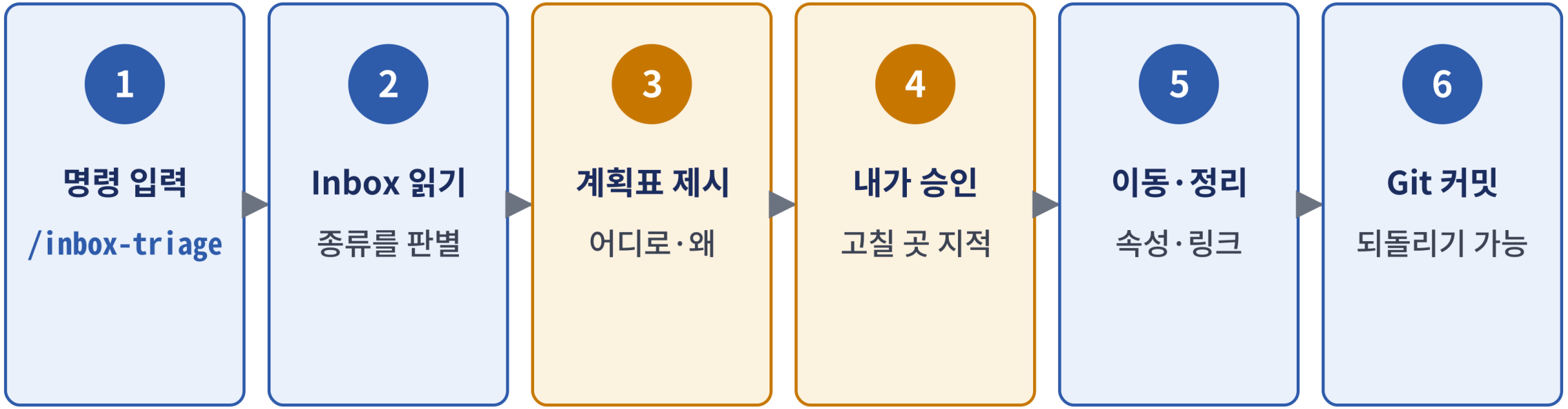
명령 네 개로 볼트 정리하기

/inbox-triage → /atomize → /link → /ask

정리 명령은 계획표를 보여 주고 승인을 받는다

모든 정리 명령이 같은 순서를 따른다 — 주황 단계가 사람의 몫이다

Claude는 바로 옮기지 않고, 먼저 계획을 보여 준다



주황 = 사람이 확인하는 단계

실습 ① — Inbox의 메모 세 개를 분류시킨다

메모 내용은 무엇이든 좋다 — 강의 아이디어, 읽은 기사, 회의 메모

[입력] 명령어 · Claude Code에 입력

```
/inbox-triage
```

[출력] 결과 · 계획표 (예시)

현재 파일	type	새 경로	이유
무제.md	idea	02_Areas/teaching/ideas/	다음 차수용 착상
기사 메모.md	literature	03_Resources/Literature/papers/	기사 요약
3-2 일정.md	project	01_Projects/kma-auto-genai-3-2/	마감 있는 일

진행할까요? 고칠 곳이 있으면 말씀해 주세요.

계획이 틀리면 말로 고친다 — '기사 메모는 그냥 Inbox에 둬'

실습 ② — 긴 자료를 개념 노트로 쪼갠다

강의 노트·논문·긴 메모 하나에서 다시 쓸 개념과 케이스를 뽑는다

[입력] 명령어 · Claude Code에 입력

```
/atomize 00_Inbox/day1-lecture-notes.md
```

[출력] 결과 · 계획표 (예시)

단위	종류	신규/기존	파일명
CASE·SDV	개념	신규	case-sdv.md
도요타 안돈	개념	신규	tps-andon.md
테슬라 2008	케이스	기존	tesla-2008.md (빠진 내용만 추가)

원본은 지우지 않고, 개념이 처음 나온 곳에 [[링크]]를 겁니다.

이미 있는 노트는 새로 만들지 않는다 — 같은 개념이 두 개로 갈라지지 않는다

실습 ③④ — 노트를 잇고, 내 노트에게 묻는다

/link는 관련 노트를 양방향으로 잇고, /ask는 볼트 안의 노트만 근거로 답한다

[입력] 명령어 · ③ 연결

```
/link tesla-2008
```

[출력] 결과 · 연결 후보

노트	관계	근거
gm-bankruptcy-2009	유사	현금 위기
case-sdv	응용	모델 S의 OTA

승인한 노트는 양쪽 related에 추가

[입력] 명령어 · ④ 질문

```
/ask 현금 위기를 다룬 케이스는?
```

[출력] 결과 · 답

```
결론: 두 건입니다.
· [[tesla-2008]] — 파산 3일 전의 겨울
· [[gm-bankruptcy-2009]] — 현금 고갈

볼트에 없는 내용은 따로 표시합니다.
```

/ask의 답에는 근거 노트 링크가 붙는다 — 눌러서 원문을 확인한다

세 가지 안전장치 덕분에 실수해도 되돌릴 수 있다

키트의 CLAUDE.md에 이미 들어 있는 규칙이다

계획표 먼저

5개 이상 바꿀 때

옮기기·쪼개기 전에 표로 보여 주고 승인을 받는다

삭제 금지

대신 Archive로

Claude는 노트를 지우지 않는다. 지우는 일은 사람이 한다

작업마다 커밋

Git 기록

'방금 작업 되돌려 줘' 한마디로 원래대로 돌아간다

Manual 권한 모드까지 더하면 사람이 모르는 변경은 일어나지 않는다

결과가 마음에 안 들면 말로 되돌린다

Git 기록 덕분에 마지막 작업 직전으로 돌아갈 수 있다

[입력] 프롬프트 · Claude Code

```
방금 /inbox-triage 작업을 되돌려 줘.
되돌리기 전에 어떤 파일이
원래대로 돌아가는지 보여 줘.
```

[출력] 결과 · 되돌리기 계획

```
되돌릴 커밋: inbox-triage: 3개 분류
원래 자리로: 무제.md, 기사 메모.md,
             3-2 일정.md

→ 승인하시면 git revert를 실행합니다.
```

되돌리기도 기록으로 남는다 — 되돌린 것을 다시 되돌릴 수도 있다

자주 막히는 곳

Level 2에서 자주 막히는 곳은 네 군데다

막히면 이 표부터 확인한다

Q1	/ 를 쳐도 명령이 안 보인다 → 볼트 맨 위 폴더를 열었는지 확인	**폴더 위치**
Q2	한 과정만 다루고 싶다 → 그 과정의 하위 폴더를 연다	**작업 범위**
Q3	'Git is required' 메시지 → Windows는 Git 설치 후 앱 재시작	**Git**
Q4	규칙을 안 지킨다 → 'CLAUDE.md 다시 읽고 해 줘'라고 말한다	**규칙**

Level 2를 마치며 — 세 가지를 기억한다

다음 강의는 노트의 상태(status)로 볼트를 운영한다

1

Claude Code는 볼트 폴더를 직접 읽고 고친다

옵시디언과 같은 폴더를 보므로, 고친 결과가 화면에 바로 나타난다.

2

규칙은 CLAUDE.md, 반복 작업은 명령으로

/inbox-triage · /atomize · /link · /ask 네 개로 일상 정리가 끝난다.

3

계획표·삭제 금지·Git이 안전장치다

Claude가 먼저 보여 주고, 사람이 승인하고, 잘못되면 되돌린다.

다음 강의 — Level 3: 상태·템플릿·대시보드로 볼트를 운영한다